Feature Selection vs Classification

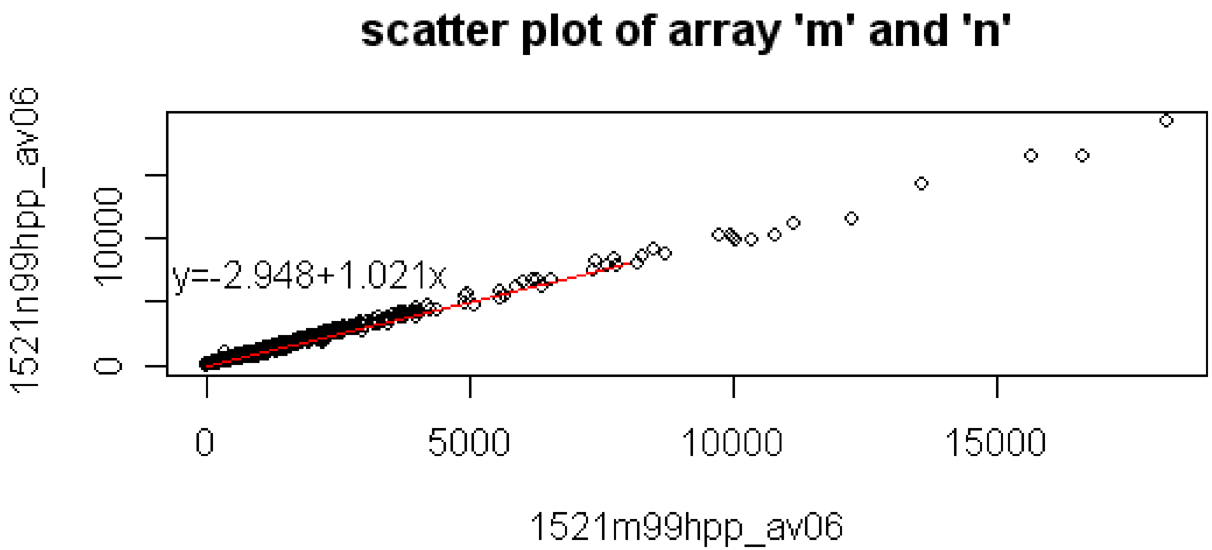
Check Read-in Data

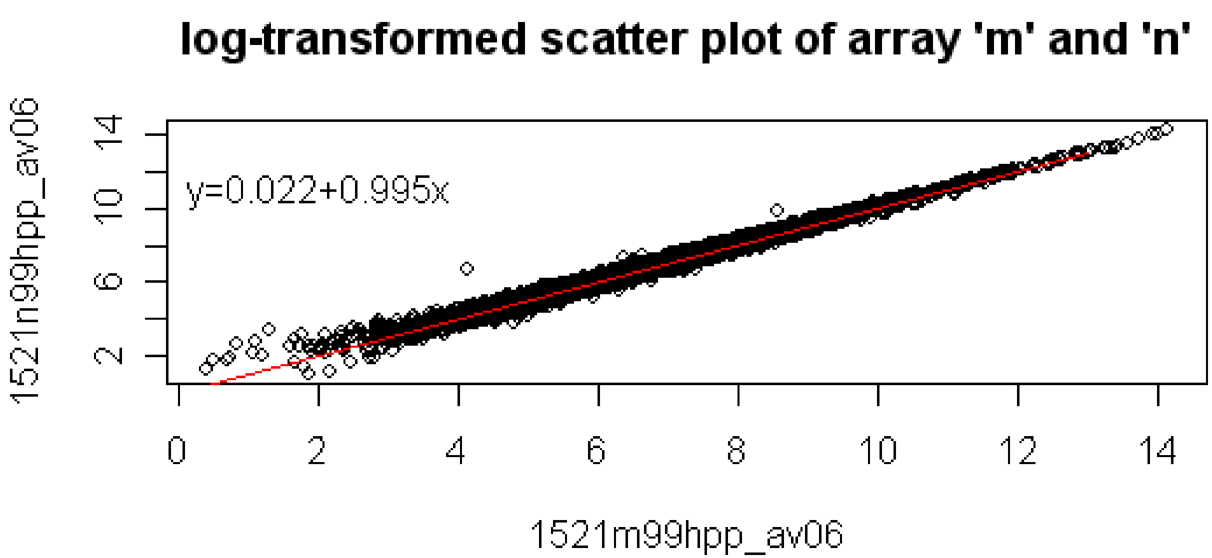
```
> data<-read.delim("MLL_train.txt")
> dim(data)
> data[1:3,]
> rownames(data)<-data[,1]
> data1<-read.delim("MLL_train.txt",row.names=1)
> (1:nrow(data))[row.names(data)=="AFFX-DapX-3_at"]
> grep("AFFX",rownames(data))

#### correlation matrix ####
> data2<-data1[,-1]
> cor1<-cor(data2)
> cor2<-cor(t(data2))
```

Scatter Plot

```
> data <- read.delim("a1_expression.txt", row.names=1)
> par(mfrow=c(2,1))   ## par: specify graph parameters before figure is plotted
> plot(data[,1], data[,2], xlab="1521m99hpp_av06", ylab="1521n99hpp_av06")
> lines(c(0, 8000), c(0, 8000), col=2)     ## col (color of the line)
> a<-lm(data[,2]~data[,1])    ## lm: fit a linear model
> text(2000, 7000, paste("y=", round(a[[1]][1], 3), "+", round(a[[1]][2], 3), "x",
    sep=""))
> title("scatter plot of array 'm' and 'n'")

###   log2 transformation  ###
> plot(log2(data[,1]), log2(data[,2]), xlab="1521m99hpp_av06",
    ylab="1521n99hpp_av06")
> lines(c(0, 13), c(0, 13), col=2)
> a<-lm(log2(data[,2])~log2(data[,1]))
> text(2, 11, paste("y=", round(a[[1]][1], 3), "+", round(a[[1]][2], 3), "x", sep=""))
> title("log-transformed scatter plot of array 'm' and 'n'")
```

Scatter Plot (cont'd)

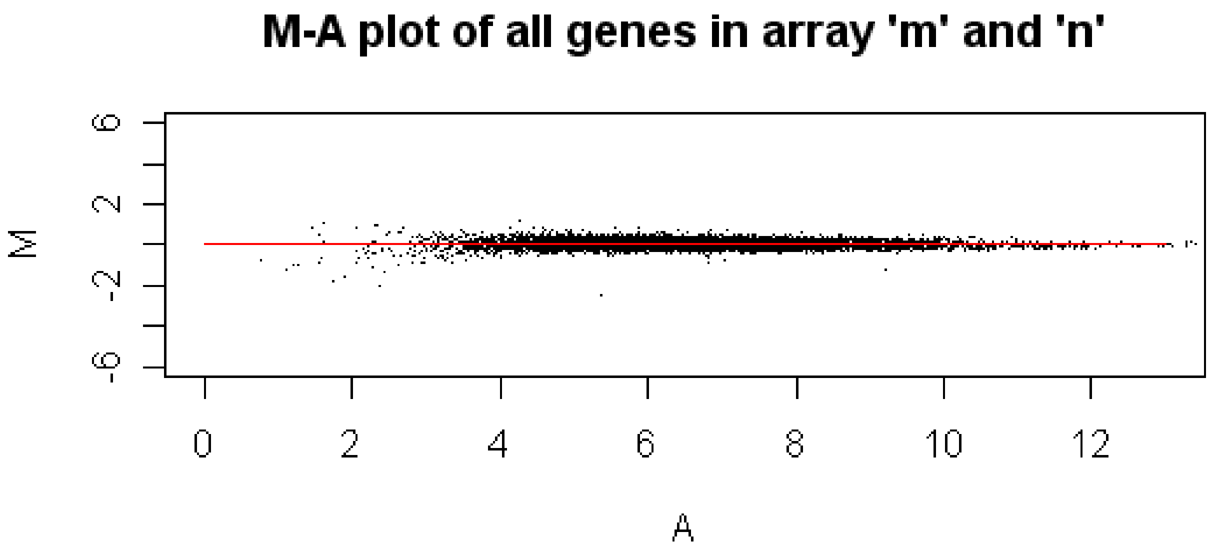


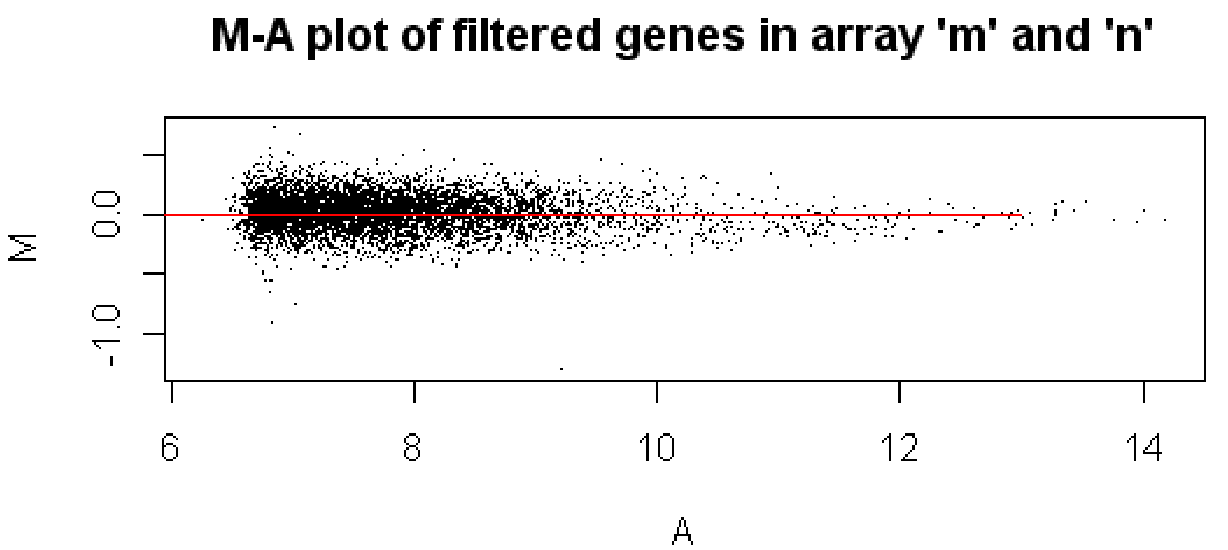


MA Plot

```
> A<-(log2(data[,1])+log2(data[,2]))/2
> M<-(log2(data[,1])-log2(data[,2]))
> plot( c(0, 13), c(-6, 6), type="n", xlab="A", ylab="M")  ### type (type of plot)
> points(A, M, pch=".")   ###   pch (types of points plotted)
> lines(c(0, 13), c(0,0), col=2)
> title("M-A plot of all genes in array 'm' and 'n'")

### filter out genes with average intensity less than 100) ###
> row.average<-apply(data, 1, mean)
> index<-(1:nrow(data))[row.average>=100]
> new.data<-data[index,]
> A<-(log2(new.data[,1])+log2(new.data[,2]))/2
> M<-(log2(new.data[,1])-log2(new.data[,2]))
> plot( c(min(A), max(A)), c(min(M), max(M)), type="n", xlab="A", ylab="M")
> points(A, M, pch=".")
> lines(c(0, 13), c(0,0), col=2)
> title("M-A plot of filtered genes in array 'm' and 'n'")
```

MA Plot (cont’d)





Detecting Differentially Expressed (DE) Genes

Select Differentially Expressed (DE) Genes

	Tumor	Tumor	Tumor	Tumor	Normal	Normal	Normal	Normal	
31308_at	21.0199	29.1547	17.9257	20.3766	19.8673	18.4821	17.9005	20.863	No
31309_r_at	46.0512	48.7559	43.1192	46.5921	25.2423	33.0099	30.2182	27.3594	Yes
31310_at	20.3716	27.6846	20.7468	18.5927	16.1071	15.4484	16.9989	16.1746	Yes
31311_at	75.6513	94.4134	80.6328	84.4216	71.8248	69.553	78.4236	71.5484	??
31312_at	97.5175	154.163	90.5806	118.928	115.495	130.89	100.678	89.8753	No
31313_at	50.9551	58.7498	54.0995	46.8968	61.6732	62.3931	64.7219	57.7332	??
31314_at	52.2138	62.3064	59.9553	54.8983	77.118	61.0678	84.1336	82.37	Yes
31315_at	315.543	252.801	204.426	265.601	224.804	225.89	139.36	177.225	
31316_at	12.2335	12.163	8.8393	10.0476	13.2467	13.3113	12.7941	10.0831	
31317_r_at	361.66	423.547	331.67	404.61	260.041	295.872	235.307	209.306	
31318_at	19.4059	26.4248	17.1136	16.5311	12.6095	15.2638	13.262	15.527	
31319_at	159.305	120.841	120.867	117.889	124.751	122.684	116.257	123.107	
31320_at	309.203	273.927	226.194	342.061	267.247	299.116	269.536	240.244	

Which genes are differentially expressed between tumor and normal?

Bioconductor Website

Bioconductor
OPEN SOURCE SOFTWARE FOR BIOINFORMATICS

About Learn Packages Developers Search Get Started ›

Open source software for Bioinformatics

The Bioconductor project aims to develop and share open source software for precise and repeatable analysis of biological data.
We foster an inclusive and collaborative community of developers and data scientists.

Get started ›

For Users For Developers

Bioconductor Website

Bioconductor
OPEN SOURCE SOFTWARE FOR BIOINFORMATICS

About Learn Packages Developers Search Get Started

Get started

1. Install R

The current release of Bioconductor is version 3.19; it works with R version 4.4.0. Users of older R and Bioconductor must update their installation to take advantage of new features and to access packages that have been added to Bioconductor since the last release.

The development version of Bioconductor is version 3.20; it works with R version 4.4.0. More recent 'devel' versions of R (if available) will be supported during the next Bioconductor release cycle.

Step 1
Install R

1. Download the most recent version of R. The R FAQs and the R Installation and Administration Manual contain detailed instructions for installing R on various platforms (Linux, OS X, and Windows being the main ones).
2. Start the R program; on Windows and OS X, this will usually mean double-clicking on the R application, on UNIX-like systems, type "R" at a shell prompt.

2. Get the latest version of Bioconductor

Once R has been installed, get the latest version of Bioconductor by starting R and entering the following commands.

It may be possible to change the Bioconductor version of an existing installation; see the 'Changing version' section of the BiocManager vignette.

Details, including instructions to install additional packages and to update, find, and troubleshoot are provided below. A devel version of Bioconductor is available. There are good reasons for using **BiocManager::install()** for managing Bioconductor resources.

```
if (!require("BiocManager", quietly = TRUE))
    install.packages("BiocManager")
BiocManager::install(version = "3.19")
```

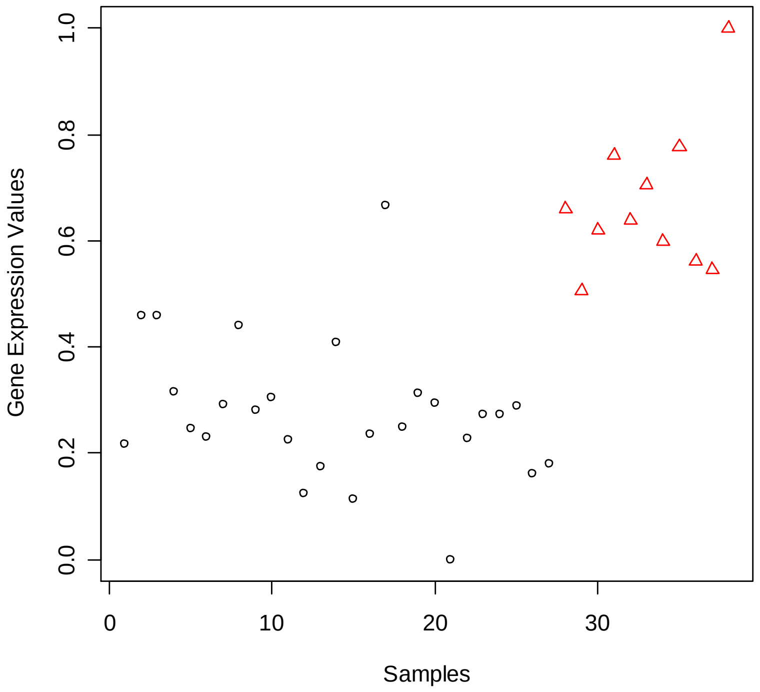
Step 2
Get Bioconductor

if (!require("BiocManager", quietly = TRUE)) install.packages("BiocManager")
BiocManager::install(version = "3.19")

https://pubmed.ncbi.nlm.nih.gov/10521349/

Golub leukemia data, 1999

```
> BiocManager::install("golubEsets")

> library(golubEsets)
> data(Golub_Train)

> GTrain<-exprs(Golub_Train)
> GTrow<-rownames(GTrain)
> GTpData<-phenoData(Golub_Train)
> GTcol<-GTpData@data$ALL.AML
> GTclass.labels<-as.numeric(GTcol)
```



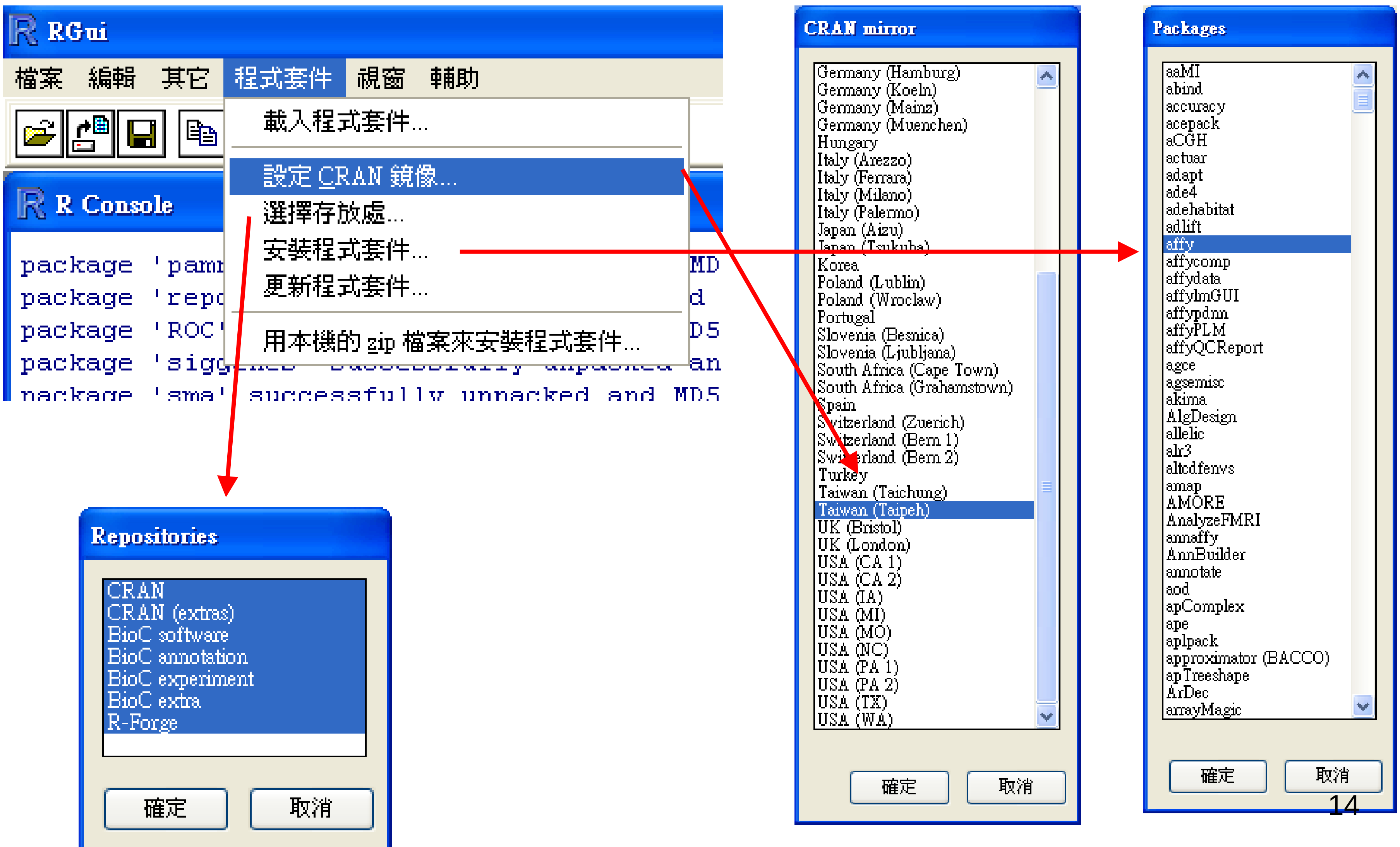

```
> x<-GTrain[2020,]
> x<- (x-min(x))/(max(x)-min(x))
> plot(x,col=GTclass.labels, pch=GTclass.labels, xlab="Samples",
  ylab="Gene Expression Values")
```

```
> t.test.R<-t.test(x[which(GTclass.labels==1)],
  x[which(GTclass.labels==2)])
> str(t.test.R)
> t.test.R$p.value

> w.test.R<-wilcox.test(x[which(GTclass.labels==1)],
  x[which(GTclass.labels==2)])
> str(w.test.R)
> w.test.R$p.value
```

Install Packages

- Install packages from CRAN
  - use CRAN website.
  - use R toolbar.
- Install packages from Bioconductor
  - use Bioconductor website.
  - use R toolbar.
- Install packages from local Zip files
  - download package files from CRAN、Bioconductor
- Information on packages

Install Packages Use R Toolbar



Download Affy Package from Bioconductor Website

Bioconductor
OPEN SOURCE SOFTWARE FOR BIOINFORMATICS

About Learn Packages Developers Search

Installation

To install this package, start R (version "4.4") and enter:

```
if (!require("BiocManager", quietly = TRUE))
    install.packages("BiocManager")

BiocManager::install("affy")
```

For older versions of R, please refer to the appropriate Bioconductor release.

Documentation

To view documentation for the version of this package installed in your system, start R and enter:

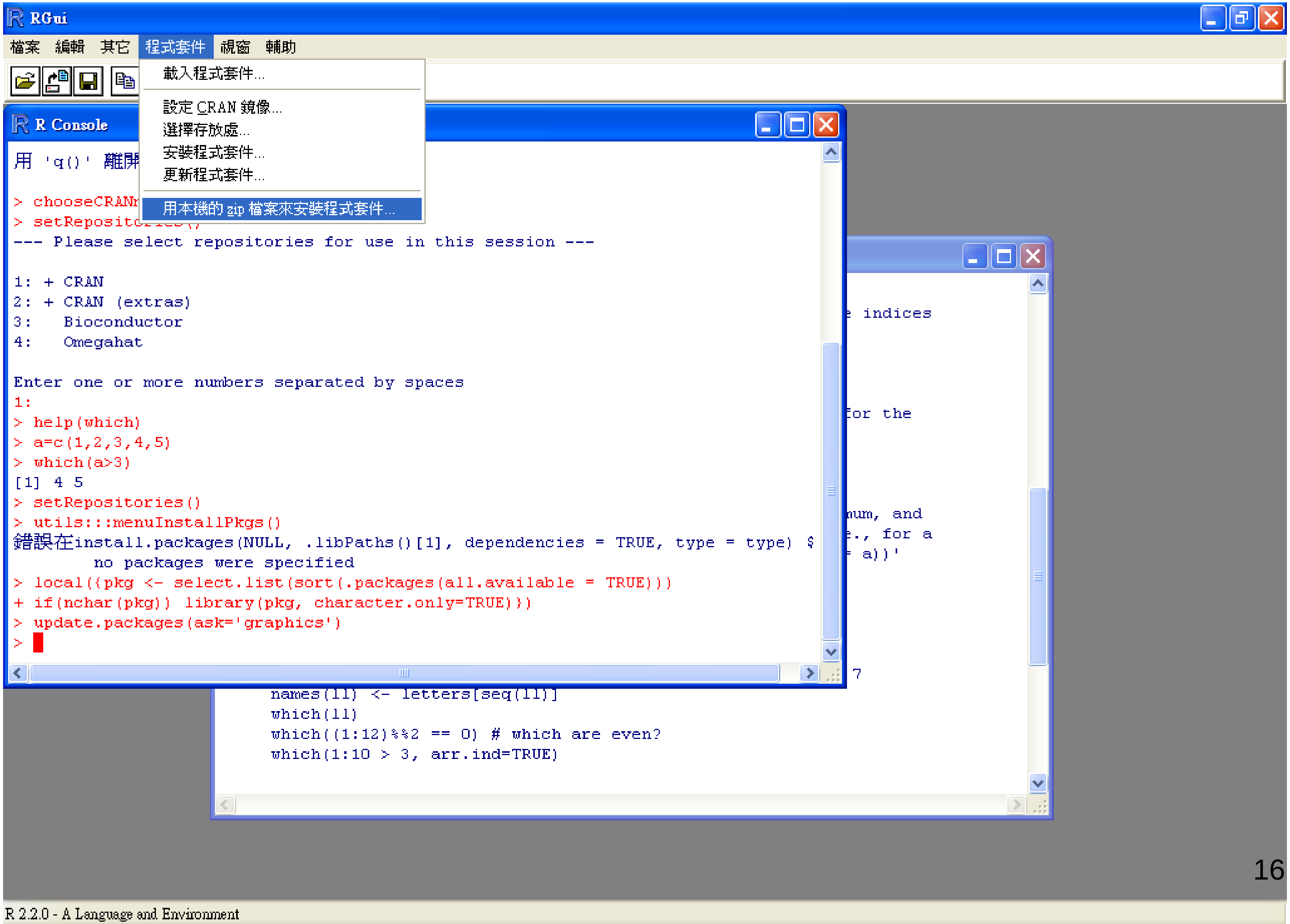
```
browseVignettes("affy")
```

Package Archives

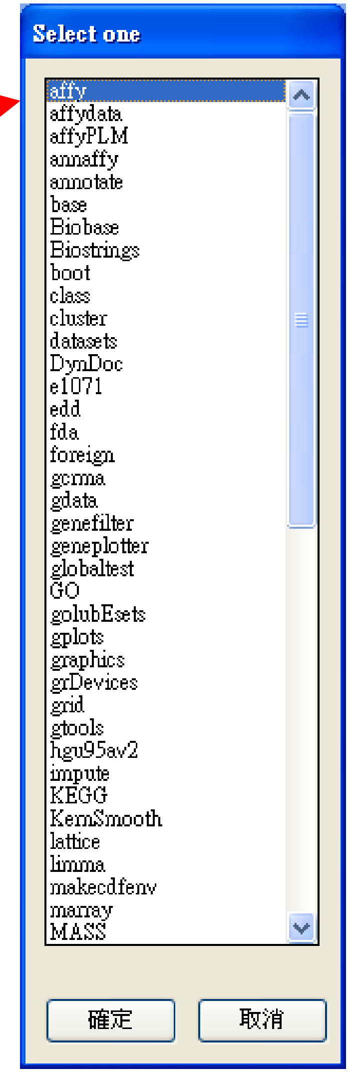
Follow Installation instructions to use this package in your R session.

Source Package	affy_1.82.0.tar.gz
Windows Binary	affy_1.82.0.zip (64-bit only)
macOS Binary (x86_64)	affy_1.82.0.tgz
macOS Binary (arm64)	affy_1.82.0.tgz
Source Repository	git clone https://git.bioconductor.org/packages/affy
Source Repository (Developer Access)	git clone git@git.bioconductor.org:packages/affy
Bioc Package Browser	https://code.bioconductor.org/browse/affy/
Package Short Url	https://bioconductor.org/packages/affy/
Package Downloads Report	Download Stats

Install Packages from Local Zip Files



Loading Package




```
>library(affy)

>help(package="affy")
>data(package="affy")
>?SpikeIn
>data(SpikeIn)
```

Keep Installed Packages in a New R Version

- Uninstall the old version
- Install the new version
- Copy the sub-directory "library" in the old-version directory to the sub-directory "library" in the new-version directory

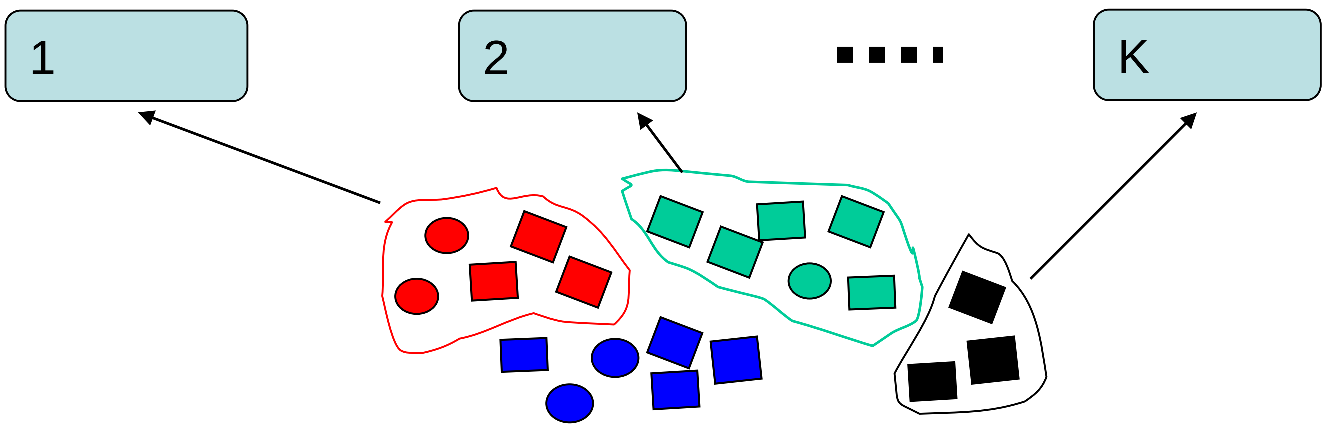
```
>update.packages(checkBuilt=T, ask=F)
```

Classification

Basic Principles of Discrimination

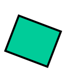
- Each object associated with a class label (or response) $Y \in \{1, 2, \ldots, K\}$ and a feature vector (vector of predictor variables) of G measurements: $X = (X_1, \ldots, X_G)$

Aim: predict Y from X.



Predefined Class {1,2,…K}

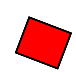
Objects



Y = Class Label = 2

X = Feature vector {colour, shape}

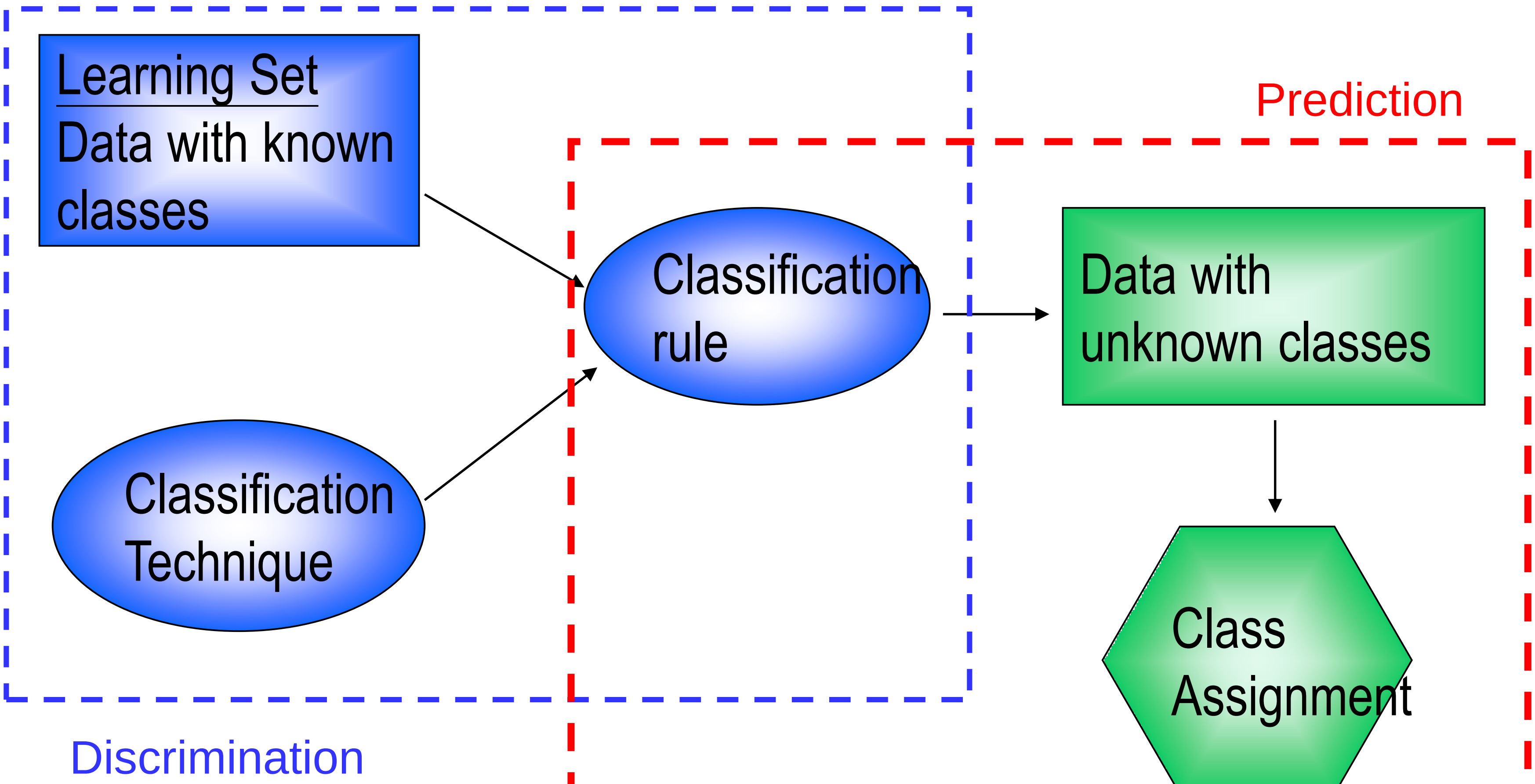
Classification rule ?



X = {red, square}

Y = ?

Discrimination and Allocation

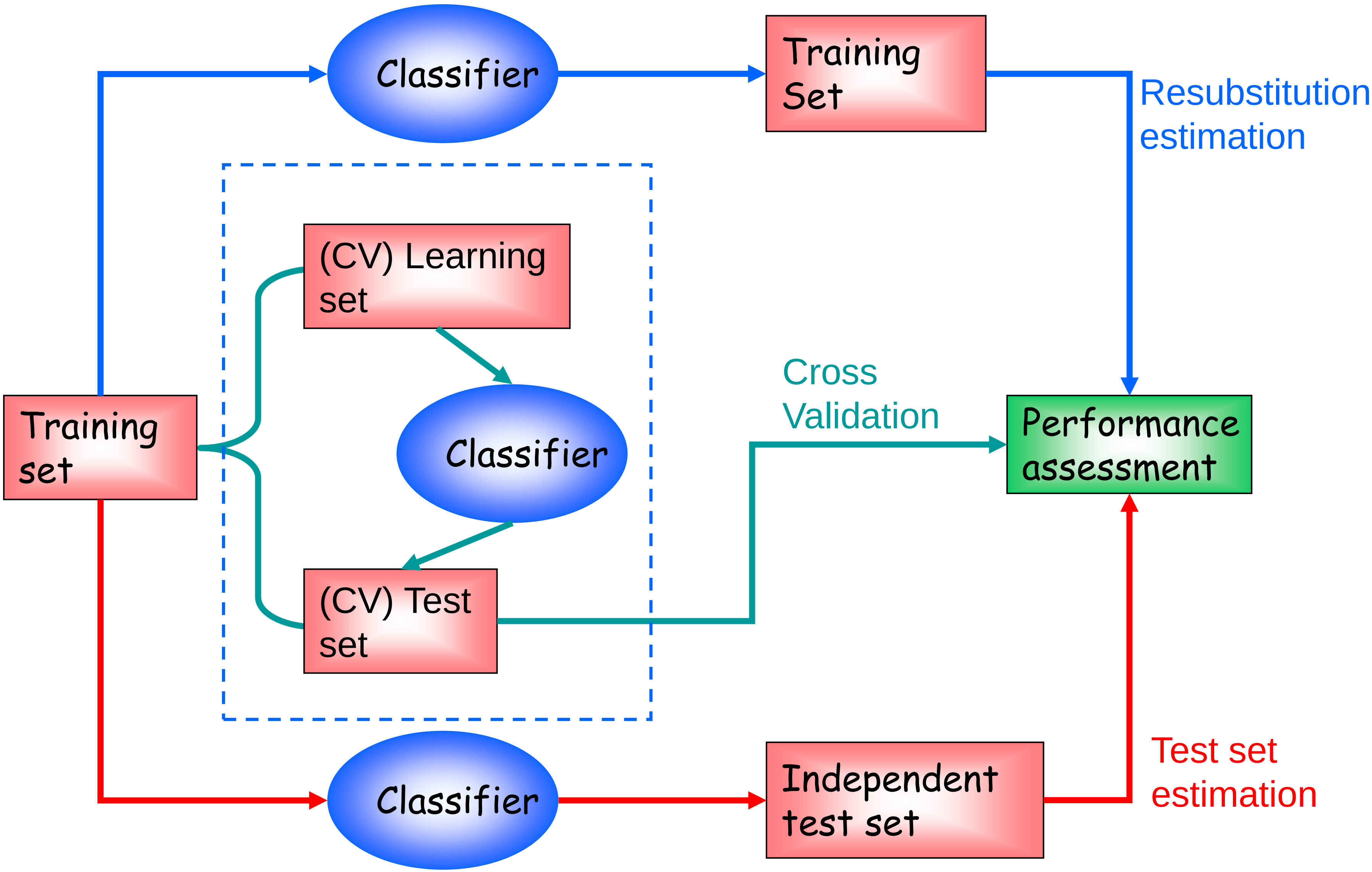


Performance Assessment

- Any classification rule needs to be evaluated for its performance on the future samples.

- Assessing performance of the classifier based on
  - Cross-validation
  - Independent testing on future dataset

Diagram of Performance Assessment

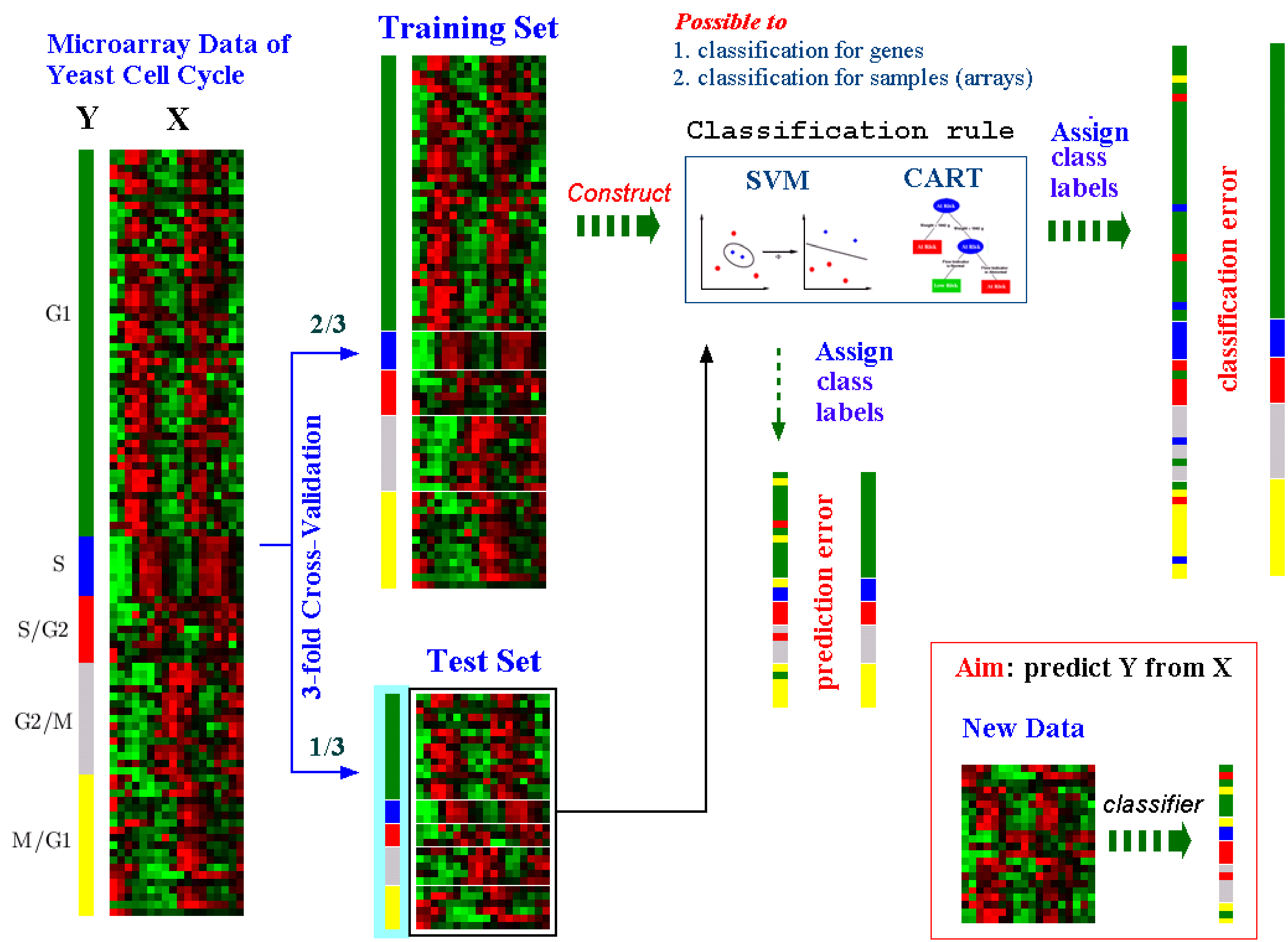


Cross-Validation

- k-fold cross-validation (CV) estimation: Cases in learning set randomly divided into k subsets of (nearly) equal size.
  - Build classifiers by leaving one set out
  - Compute test set error rates on the left out set and averaged
  - Total up accuracy
- Leave-one-out cross validation (LOOCV). (Special case for k=n).
  - Works well for stable classifiers (k-NN, LDA, SVM)

1.Test	2.Train	3.Train	4.Train	5.Train
1.Train	2.Test	3.Train	4.Train	5.Train
1.Train	2.Train	3.Test	4.Train	5.Train
1.Train	2.Train	3.Train	4.Test	5.Train
1.Train	2.Train	3.Train	4.Train	5.Test

3-fold Cross-Validation



Confusion Matrix

Leukemias

Microarray		ALL	AML	
	ALL	a	b	a+b
	AML	c	d	c+d
		a+c	b+d	

	predicted as positive	predicted as negative
positive	TP	FN
negative	FP	TN

Disease

Test (Diagnosis)		+	-	
	+	a	b	a+b
	-	c	d	c+d
		a+c	b+d	

01010000100110011111000 → Observed
11000011100011111110011 → Predicted

TP FP FN TN

a: # True Positives (TP) ,
b: # False Positives (FP) ,
c: # False Negatives (FN) ,
d: # True Negatives (TN)

Sensitivity = a/(a+c) = TP/(TP+FN)

Specificity = d/(b+d) = TN/(FP+TN)

False Positive Rate = b/(b+d)
= FP/(FP+TN)
= 1 - Specificity

False Negative Rate = c/(a+c)
= FN/(TP+FN)
= 1 - Sensitivity

Accuracy = (a+d)/(a+b+c+d)
= (TP+TN)/(TP+FP+FN+TN)

Classification Methods Available in Bioconductor

"MLInterfaces" package

This package is meant to be a unifying platform for all machine learning procedures (including classification and clustering methods). Useful but use of the package easily becomes a black box!!

- Linear and quadratic discriminant analysis: "ldaI"
- KNN classification: "knnI"
- Bagging and AdaBoosting: "baggingI" and "logitboostI"
- Random forest: "randomForestI"
- Support Vector machines: "svmI"
- Artifical neural network: "nnetI"

Classification Methods Available in R Packages

Logistic regression:
“glm” with parameter “family=binomial().

Linear and quadratic discriminant analysis:
“lda” and “qda” in “MASS” package

DLDA and DQDA: “stat.diag.da” in “sma” package

KNN classification: “knn” in “class”package

CART: “rpart” package

Bagging and AdaBoosting: “adabag” package

Random forest: “randomForest” package

Support Vector machines: “svm” in “e1071” package

Nearest shrunken centroids: “pamr” in “pamr” package

Loading Leukemia Data

```
## a data package about leukemia ##
BiocManager::install("ALL")     ## download the latest version suited to your R version.
library(ALL)   ## load the package into R
data(ALL)    ## manually load a specific data into R
ALL     ## data set “ALL”; It is an instance of “ExpressionSet” class

exprs(ALL)[1:3,]    ## get the first three rows of the data matrix
varLabels(ALL)
ALL.1<-ALL[,order(ALL$mol.bio)]    ### order samples by sample types
ALL.1$mol.bio
```

Effect on Gene Selection by Correlation Map

```
#### plot the correlation matrix of the 128 samples using all 12625 genes #####
heatmap(cor(exprs(ALL.1)), Rowv=NA, Colv=NA, scale= "none",
   labRow=ALL.1$mol.bio, labCol= ALL.1$mol.bio, RowSideColors=
   as.character(as.numeric(ALL.1$mol.bio)), ColSideColors=
   as.character(as.numeric(ALL.1$mol.bio)))

#### make a simple filtering and select genes with sd larger than 1 #####
ALL.sd<-apply(exprs(ALL.1), 1, sd)
ALL.new<-ALL.1 [ALL.sd>1, ]
ALL.new   #### 379 genes left
X11()
heatmap(cor(exprs(ALL.new)), Rowv=NA, Colv=NA, scale="none",
   labRow=ALL.new$mol.bio, labCol= ALL.new$mol.bio, RowSideColors=
   as.character(as.numeric(ALL.new$mol.bio)), ColSideColors=
   as.character(as.numeric(ALL.new$mol.bio)))

## In the correlation plot, NEG has two subgroups.
## One is B-cell (BT=B) and the other is T-cell (BT=T)
ALL$BT
```

Preprocessing of Leukemia Data

```
## select samples from two groups: BCR/ABL and NEG
bio<-which(ALL$mol.bio %in% c("BCR/ABL", "NEG"))

## select samples with B-cell, not T-cell
isb<-grep("^B", as.character(ALL$BT))

## select all the sample of B-cell in BCR/ABL or NEG
kp<-intersect(bio, isb)
ALL.2<-ALL[, kp]

tmp<-ALL.2$mol.bio == "BCR/ABL"
tmp<-ifelse(tmp, "BCR/ABL", "NEG")
pData(ALL.2)$bcrabl<-factor(tmp)
ALL.2$bcrabl
```

Machine Learning Tools in R

```
## software package of a unifying interface for machine learning methods ##
BiocManager::install("MLInterfaces")
library(MLInterfaces)        ## load in the library

#### Training and test set cross-validation
####  take the first 40 samples as training set and test on the remaining 39 samples
t.stat<-function(x, group)
{
    t.test(x[group=="BCR/ABL"], x[group=="NEG"])$statistic
}
a<-apply(exprs(ALL.2)[, 1:40], 1, t.stat, group=ALL.2$bcrabl[1:40])

#### select the top 50 genes with the larges standard deviations
index<-order(abs(a))[length(a):(length(a)-50+1)]

#### now we start with knn ######
help(MLearn)
args(MLearn)
all.knn<- MLearn(bcrabl~., ALL.2[index,], knnI(k=3,l=1), 1:40)
#### tabulate predictions and underlying truth for the 39 predicted samples
confuMat(all.knn)
```

	predicted	
given	BCR/ABL	NEG
BCR/ABL	13	3
NEG	2	21

Machine Learning Tools in R (cont'd)

```
#### SVM ####
all.svm<- MLearn(bcrabl~., ALL.2[index,], svmI, 1:40)
confuMat(all.svm)
```

given \ predicted	BCR/ABL	NEG
BCR/ABL	14	2
NEG	8	15

```
### SVM in e1071 library ####
BiocManager::install("e1071")
library("e1071")

### training and test data ###
x_train<-t(exprs(ALL.2[index,1:40]))
y_train<-ALL.2$bcrabl[1:40]
x_test<-t(exprs(ALL.2[index,41:79]))
y_test<-ALL.2$bcrabl[41:79]

all.svm1<-svm(x_train, y_train)
summary(all.svm1)
```

```
###  test with train data
pred <- predict(all.svm1, x_train)
table(pred, y_train)
```

pred \ y_train	BCR/ABL	NEG
BCR/ABL	21	1
NEG	0	18

```
pred <- predict(all.svm1, x_test)
table(pred, y_test)
```

pred \ y_test	BCR/ABL	NEG
BCR/ABL	14	8
NEG	2	15

Machine Learning Tools in R (cont'd)

```
##### structured leave-one-out cross validation #####
##### note the gene selection here already used all samples
##### the cross validation errors are over-optimistic

## knn ##
cvloo.knn<-MLearn(bcrabl~., ALL.2[index,], knn.cvI(k=3,l=1), 1:79)
confuMat(cvloo.knn)
```

```
              predicted
given          BCR/ABL NEG
  BCR/ABL          34    3
  NEG               6   36
```

HW4

- In file MLL_train.txt, it contains the two information:
  - Three types of samples: ALL (20 samples), MLL (17 samples), AML (20 samples)
  - 12582 sets of gene expression values (Assume data are normalized)
- Please obey the following rules and finish the HW:
  - **Remove genes with the name “AFFX…”**
  - **Normalize data by each row (let values b/w 0~1)**
  - **Find the top 10 differentially expressed genes using t-test**
    - Condition 1: mean(group1) > mean(others)
    - Condition 2: mean(group2) > mean(others)
    - Condition 3: mean(group3) > mean(others)
  - **Use SVM to execute the classification process**
    - Conditions: Training data are also used as testing data
    - **Make the confusion matrix** from the classification results
    - A 3-class classification problem using all genes
    - Three 2-class classification problems, each case using all genes
    - Three 2-class classification problems, each case using 10 selected genes (refer to above)
    - A 3-class classification problem using the combination of 30 selected genes